

Conecta 4 con Kotlin

Aprende los fundamentos del desarrollo profesional construyendo un clásico.

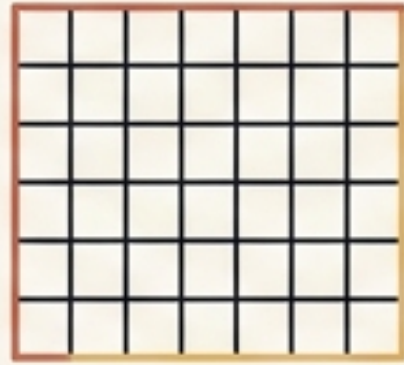


Los 5 Pilares de un Código Sólido



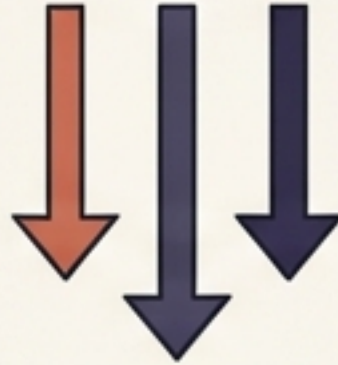
Enumeraciones Avanzadas

Definir datos exactos.



Colecciones Mutables

Construir la arena de juego.



Control de Flujo

Simular la física y las reglas.



Seguridad contra Nulos

Proteger el sistema del usuario.



Arquitectura de Funciones

Organizar la lógica como un profesional.

Bloque 1: Definiendo las Piezas con Enums



Más que simples etiquetas: son objetos con propiedades reales.

```
enum class Ficha(val simbolo: String) {  
    VACIO(". "),  
    JUGADOR("$ROJO X $RESET"),  
    MAQUINA("$AMARILLO O $RESET");  
}
```

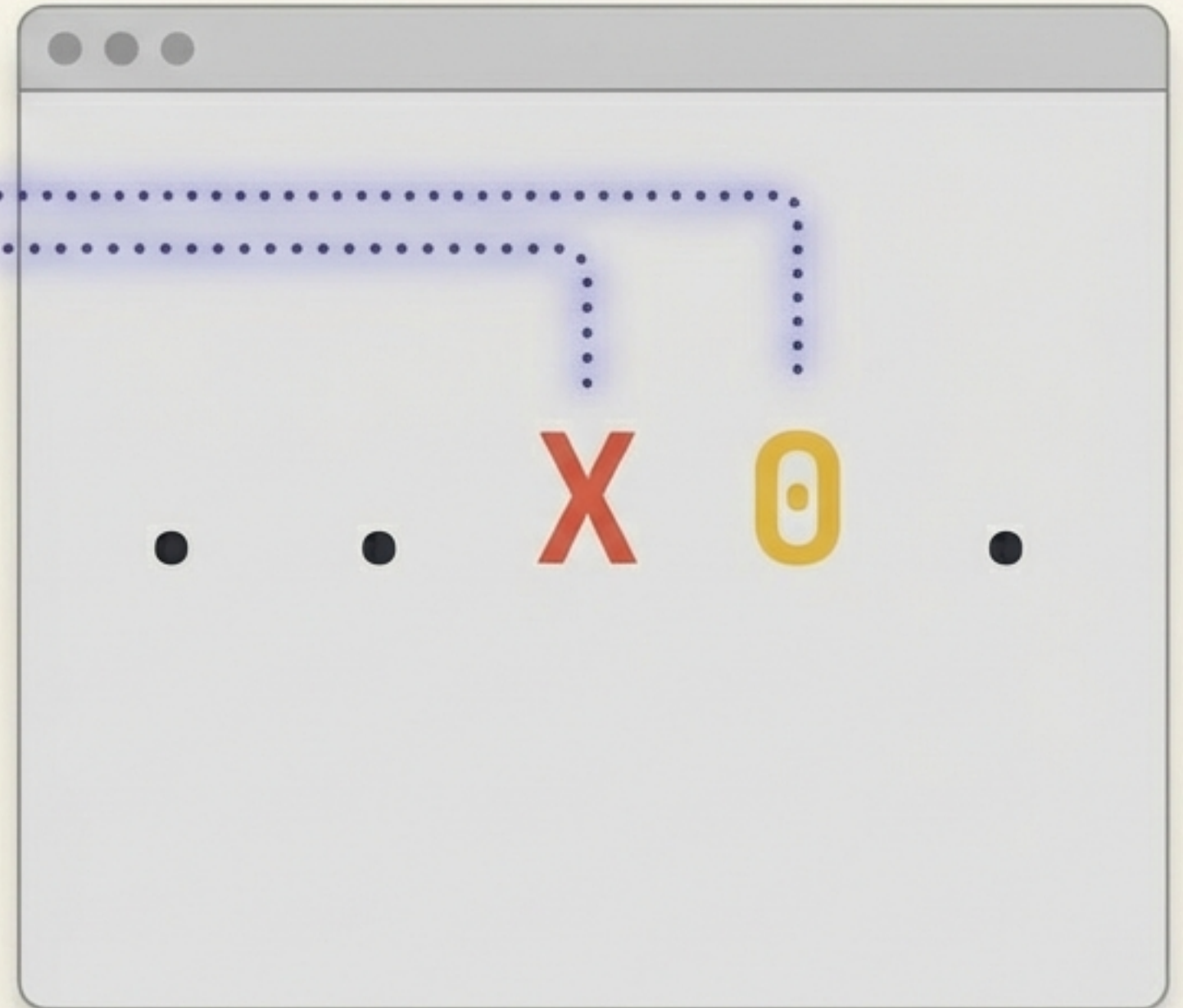
Integramos códigos RGB directamente en el estado de la ficha.

El Poder de Sobrescribir toString()

```
// Dentro del enum class Ficha  
override fun toString(): String = simbolo
```

Al sobrescribir este método, el objeto sabe cómo imprimirse a sí mismo.

Cero lógica extra en tu función main.
Imprimes la ficha y el color aparece mágicamente.



Bloque 2: Construyendo la Arena de Juego

```
val tablero = MutableList(FILAS) {  
    MutableList(COLUMNAS) { Ficha.VACIO }  
}
```

La lista principal
representa las filas.

Cada fila contiene otra lista
para las columnas.

Inicializamos cada hueco
con nuestro Enum.

Ficha.VACIO	Ficha.VACIO	Ficha.VACIO	Ficha.VACIO	Ficha.VACIO	Ficha.VACIO	Ficha.VACIO
Ficha.VACIO	Ficha.VACIO	Ficha.VACIO	Ficha.VACIO	Ficha.VACIO	Ficha.VACIO	Ficha.VACIO
Ficha.VACIO	Ficha.VACIO	Ficha.VACIO	Ficha.VACIO	Ficha.VACIO	Ficha.VACIO	Ficha.VACIO
Ficha.VACIO	Ficha.VACIO	Ficha.VACIO	Ficha.VACIO	Ficha.VACIO	Ficha.VACIO	Ficha.VACIO
Ficha.VACIO	Ficha.VACIO	Ficha.VACIO	Ficha.VACIO	Ficha.VACIO	Ficha.VACIO	Ficha.VACIO
Ficha.VACIO	Ficha.VACIO	Ficha.VACIO	Ficha.VACIO	Ficha.VACIO	Ficha.VACIO	Ficha.VACIO

Listas dentro de Listas

	c →						
	0	1	2	3	4	5	6
0	○	○	○	○	○	○	○
1	○	○	○	○	○	○	○
2	○	○	○	○	○	○	○
3	○	○	○	○	○	○	○
4	○	○	○	✗	○	○	○
5	○	○	○	✗	○	○	○

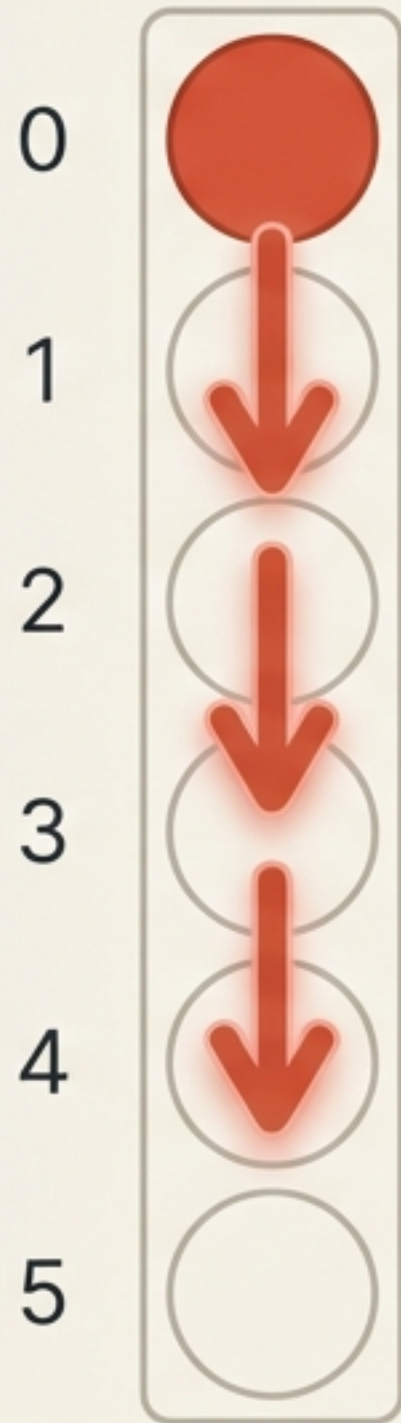
Índice de la fila
(vertical).

```
tablero[f][c] = ficha
```

Índice de la columna
(horizontal).

Al ser **MutableList**, podemos reescribir cualquier coordenada en tiempo real.

Bloque 3: Simulando Gravedad con downTo

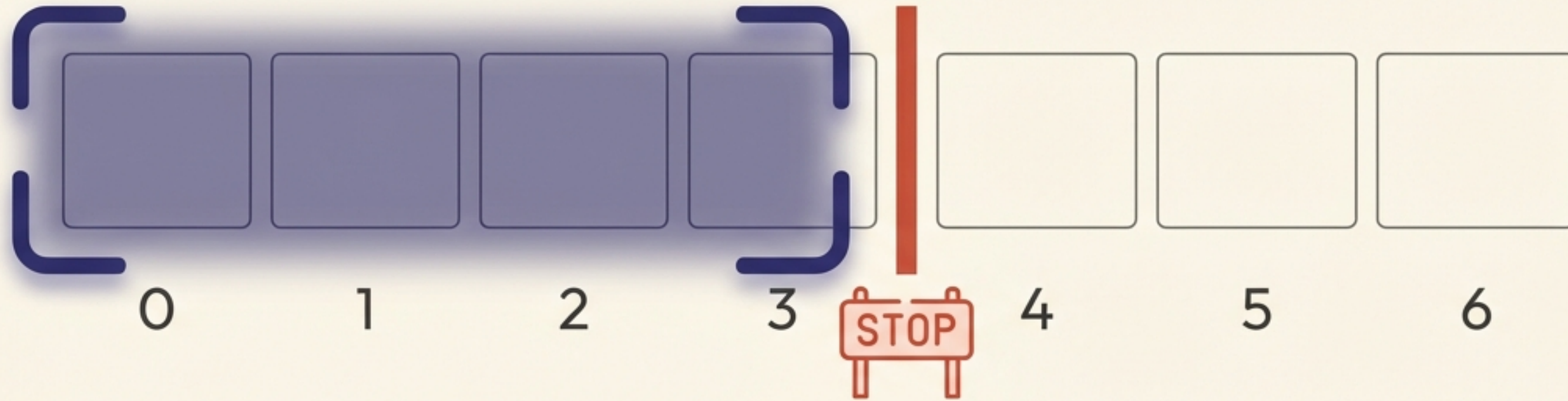


```
// Simula la gravedad cayendo hasta el fondo
for (f in FILAS - 1 downTo 0) {
    if (tablero[f]c == Ficha.VACIO) {
        // ...
    }
}
```

Empezamos desde el suelo del tablero.

Iteramos hacia atrás, subiendo fila a fila hasta encontrar el primer hueco.

Navegando el Tablero sin Estrellarse

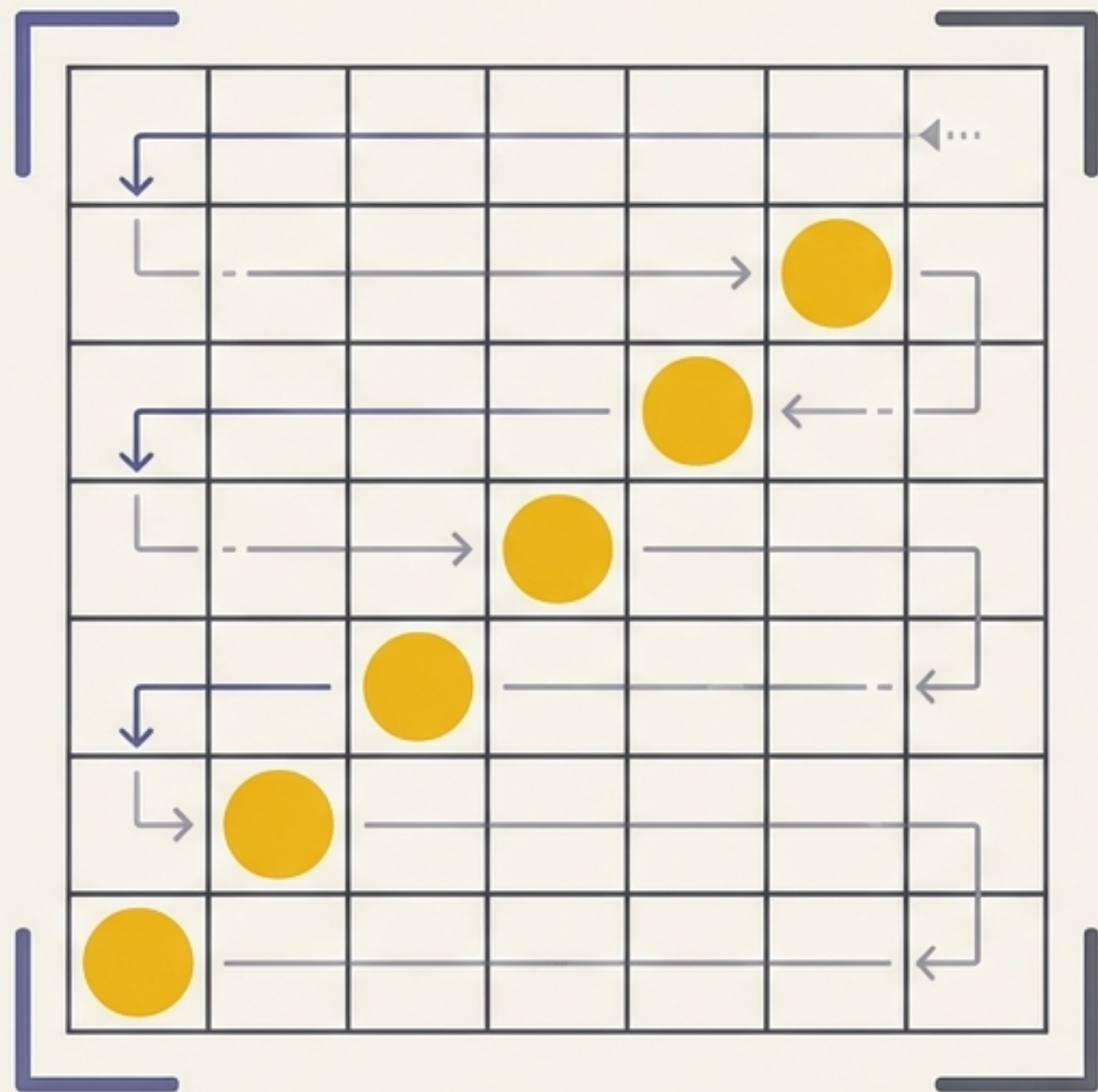


```
// Búsqueda en dirección HORIZONTAL
for (c in 0 until COLUMNAS - 3) {
    if (tablero[f][c] == ficha && tablero[f][c+1] == ficha...
}
```

Rango seguro.

Evita el temido error `IndexOutOfBounds`. Si vamos a comprobar 4 fichas hacia adelante ($c+3$), debemos detener el bucle 3 pasos antes del borde.

La Lógica de la Victoria



Bucles anidados recorren toda la matriz sistemáticamente.

```
for (f in 0 until FILAS) {  
  for (c in 0 until COLUMNAS - 3) {  
    // Comprueba las 4 posiciones simultáneamente  
    if (tablero[f][c] == ficha &&  
        tablero[f][c+1] == ficha...) return true  
  }  
}
```

Una vez que las reglas de rango están seguras, la lógica de victoria es simple matemática.

Bloque 4: Tu Escudo Contra el Caos

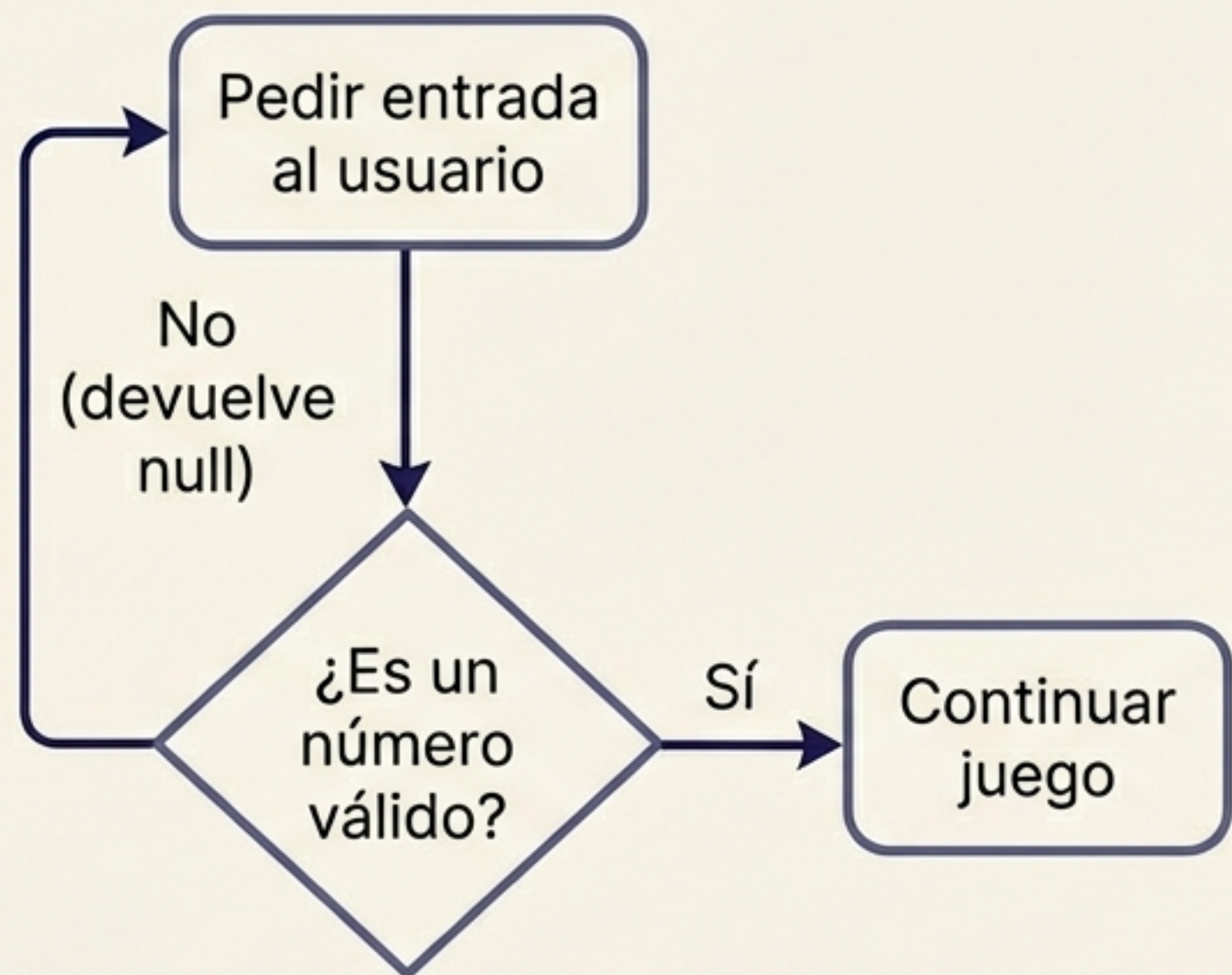
A collage of various characters, symbols, and words, including "iHola!", "xyz", "mi k", "2a3b", "man", "@error", "invalid mode", and "input!", set against a background of a globe.

```
// Entrada Segura
val input = readlnOrNull()
val numero = input?.toIntOrNull()

if (numero != null) {
    // Procesar número
} else {
    // Manejar error
}
```

El usuario nunca hace lo que esperas. Usar `readln().toInt()` provoca una caída inmediata si el texto no es un número. Kotlin ofrece una armadura nativa.

Atrapando Errores con Elegancia



```
var dificultadSeleccionada : Dificultad? = null
while (dificultadSeleccionada == null) {
    print("Opcion: ")
    val opcion = readln().trim().toIntOrNull()
    // ...
}
```

El símbolo ? indica que la variable permite valores nulos.

Si el usuario escribe letras, devuelve null en lugar de lanzar una excepción fatal. El bucle simplemente vuelve a preguntar.

Bloque 5: La Arquitectura de un Profesional



El Cerebro

Funciones que piensan y calculan sin tocar nada.



La Mano

Funciones que modifican el mundo real o el estado del juego.

El secreto de un código mantenible es separar drásticamente las funciones que piensan de las funciones que actúan.



El Cerebro: Funciones que Calculan

```
fun columnaValida(c: Int): Boolean {  
    return c >= 0 && c < COLUMNAS &&  
        tablero[0][c] == Ficha.VACIO  
}
```

Solo responde
a una pregunta.

No altera el tablero. No imprime texto. Es predecible y fácil de someter a pruebas automáticas (testing).



Las Manos: Funciones que Ejecutan

Muta el estado global.

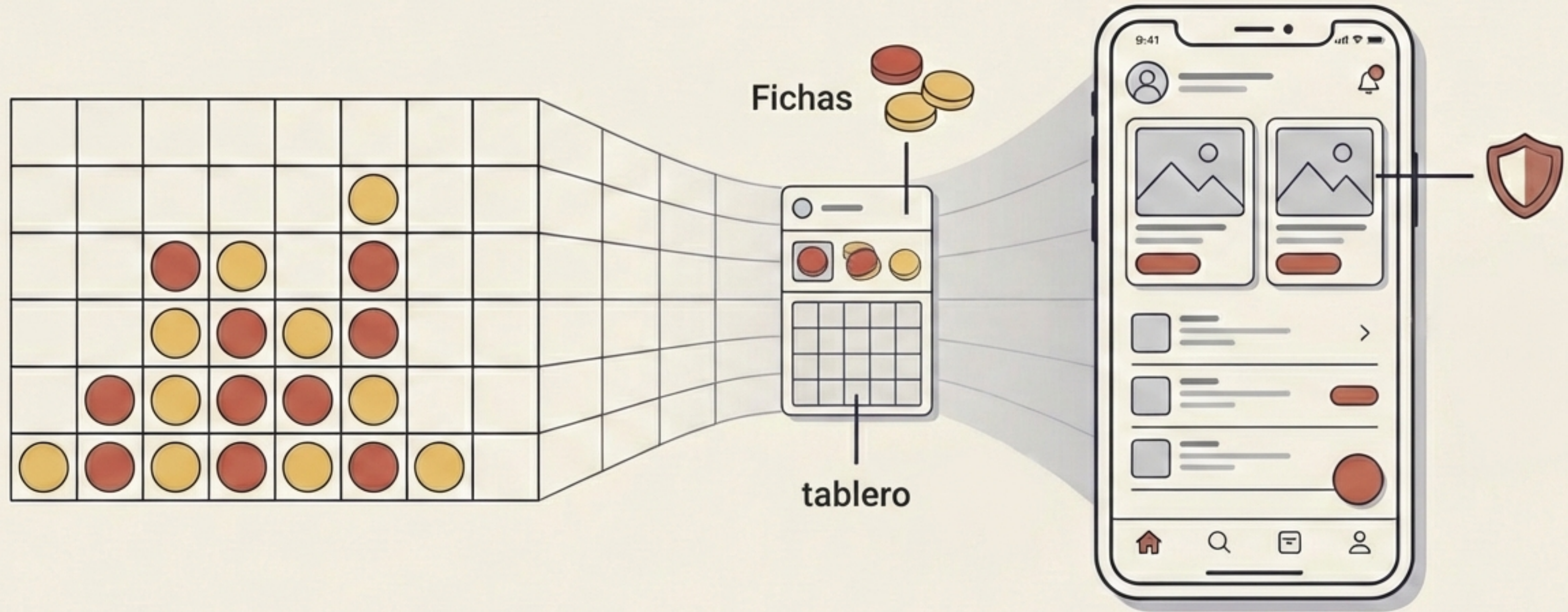
```
fun colocarFicha(c:Int, ficha:Ficha) {  
    tablero[f][c] = ficha  
}
```

Interactúa con el exterior (I/O).

```
// ...  
  
fun imprimirTablero() {  
    println("\n$AZUL+-----+$RESET")  
}
```

Estas funciones confían en que el 'Cerebro' ya validó las reglas antes de actuar.

De un Juego a Cualquier Aplicación



Dominar Enums, Colecciones Mutables, Flujo de Control, Null Safety y Arquitectura te prepara construir cualquier cosa. Ya no eres un principiante.